Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Homework 1
SF2524

Aleksi Löf & Anton Blomgren
KTH Royal Institute of Technology

November 12, 2025

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Matrix Computations for Large-scale Systems
SF2524

Contents

AI Usage Specification **1**

Problem 2 **1**
a) . . . 1
b) . . . 2
c) . . . 3
d) . . . 3

Problem 3 **4**
a) . . . 4
b) . . . 4

Problem 4 **5**
a) . . . 5
b) . . . 5
c) . . . 5

Problem 5 **6**
a) + b) . . . 6
c) . . . 6
e) + d) . . . 7

Problem 6 **8**

Problem 8 **9**
b) . . . 9
c) . . . 9
d) . . . 10
e) . . . 10
f) . . . 11

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Matrix Computations for Large-scale Systems
SF2524

AI Usage Specification

Level 2:
In this report, AI was used to check for spelling and grammar errors in text. It was also used to localize and find errors in code. It was **NOT** used to generate code or text.

Problem 2

a)

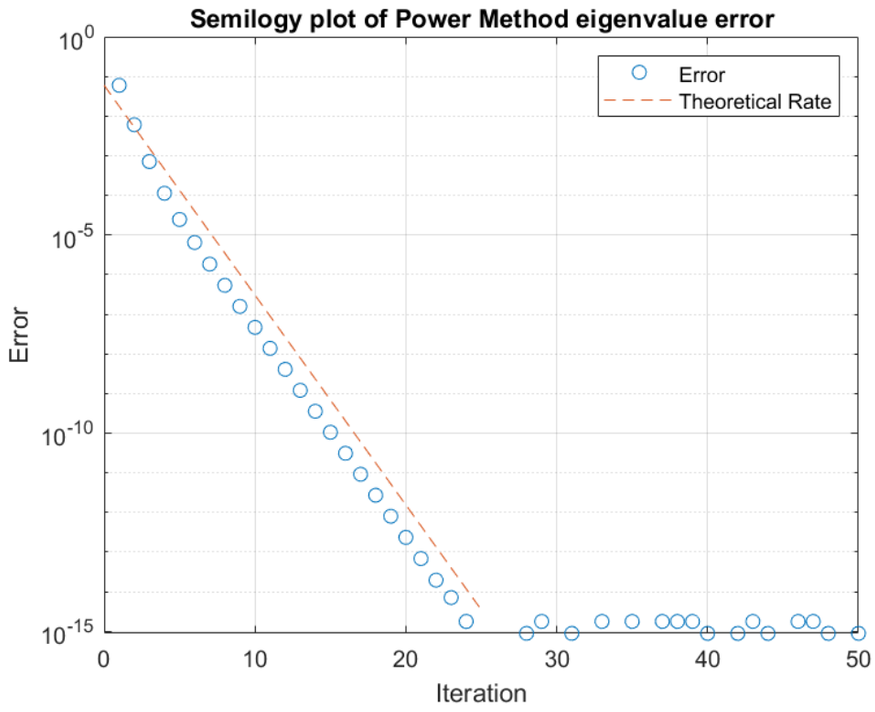


Figure 1: Power Method, $N = 50$

The ratio $\left|\frac{\lambda_2}{\lambda_1}\right|$ is the convergence rate as by Theorem 1.1.3, here $\left|\frac{\lambda_2}{\lambda_1}\right| \approx 0.5437$. The order of convergence is approximately $p \approx 1$ which is evident from Figure 1, and is also consistent with theory. From section (0.4) in the Background pdf in the course, we find the following

$$e_{k+1} \leq \alpha \cdot e_k^p, \tag{1}$$

where α is the convergence rate and p is the order of convergence. Thus, this relation with specific values is

$$e_{k+1} \leq \alpha \cdot e_k^p = \frac{|\ \lambda_2\ |}{|\ \lambda_1\ |} \cdot e_k^1 = 0.5437 \cdot e_k. \tag{2}$$

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

b)

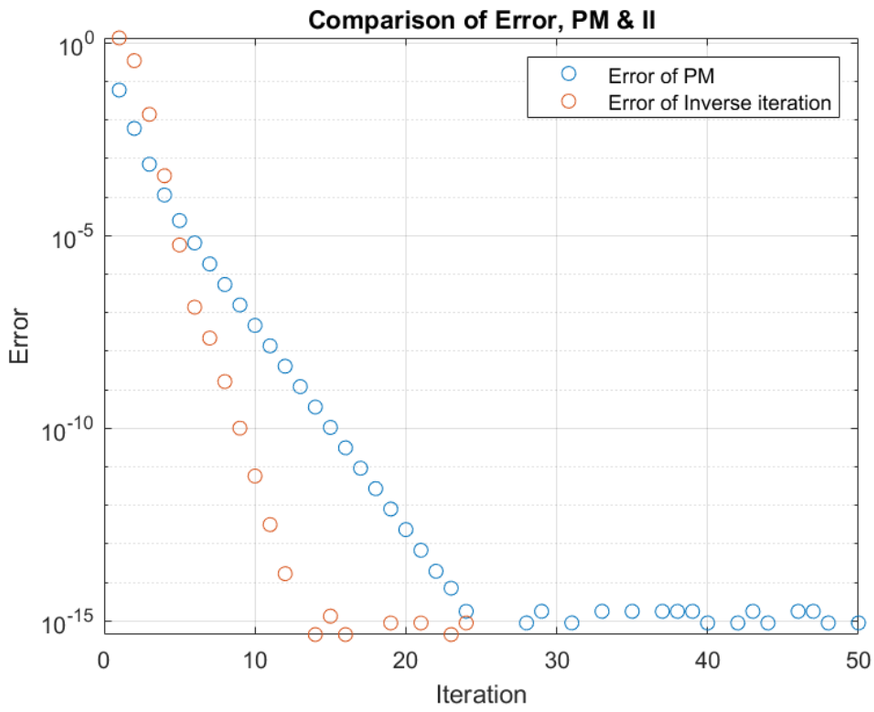


Figure 2: Comparison of Power Method and Inverse Iteration

The Inverse iteration with shift $\mu = 3$ and starting vector $x_0^T = [1, 1, 1]$ converges to $\lambda_2 = 2.6527$. In theory both the PM and Inverse iteration have a order of convergence $p = 1$, which is linear. By observing Figure 2, one can conclude that the Inverse iteration does converge faster to machine precision error level and also has linear convergence. It takes ≈ 14 iterations for the Inverse iteration while the PM needs ≈ 24.

Inverse iteration order of convergence is approximated as ≈ 1 which is consistent with theory. Also the convergence rate is calculated as ≈ 0.2267, which compared to the PM convergence rate in (2) is evidently smaller leading to faster convergence.

From (1) and from Section 1.1.5 in Block 1, we now get

$$e_{k+1} \leq \alpha \cdot e_k^p = \frac{|\,\lambda_J - \mu\,|}{|\,\lambda_K - \mu\,|} \cdot e_k^1 = 0.2267 \cdot e_k, \tag{3}$$

where λ_J is the eigenvalue closest to shift $\mu = 3$ and λ_K is second closest.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

c)

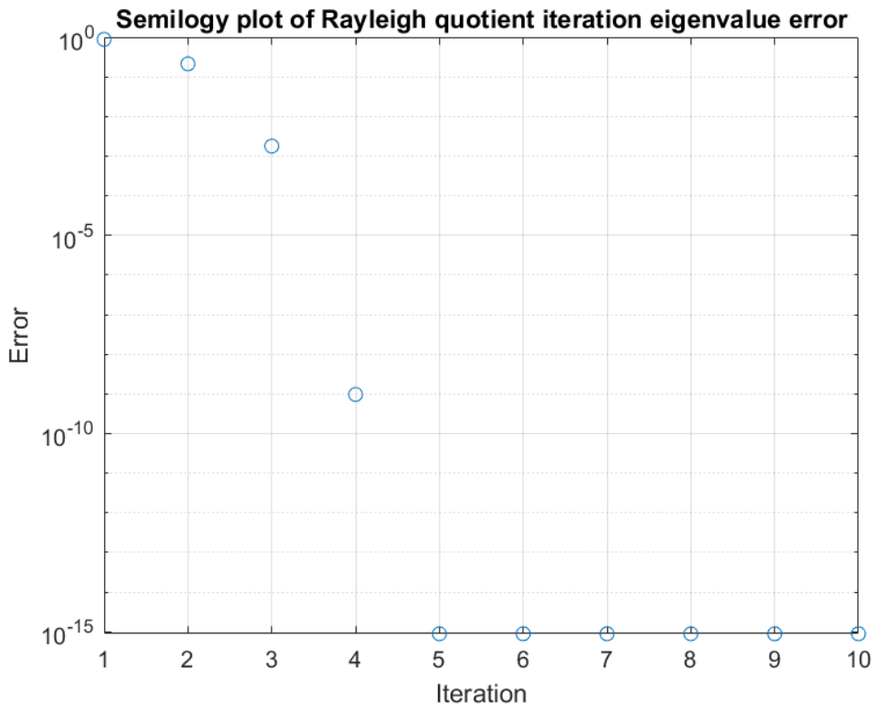


Figure 3: Rayleigh quotient iteration, $N = 10$

The observed convergence is faster than previous methods and the order of convergence was computed to be $p \approx 3$, cubic convergence. This also falls in line with theory from Section 1.1.6 in Block 1, which says that if the matrix which we want to compute the eigenvalues of is symmetric, the order of convergence should be p=3. The general case with a non-symmetric matrix is $p = 2$. It can be seen in Figure 3 that the method reaches the machine precision error level in 5 iterations.

d)

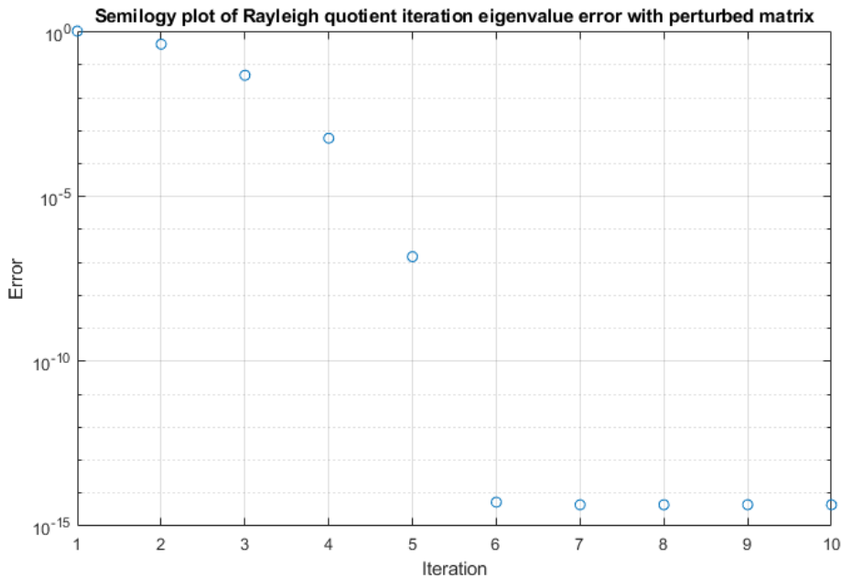


Figure 4: Rayleigh quotient iteration with perturbed matrix, $N = 10$

The matrix is no longer symmetric, so according to the theory stated in problem 2.c), the order of convergence is now $p = 2$. This can also be seen in Figure 4 where it now takes 6 iterations to get to the machine precission error level.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Problem 3

a)

	S GS, MV, nn=800		S GS, for, nn=800		mod GS, for, nn=800	
	time [s]	orth	time [s]	orth	time [s]	orth
$m = 5$	0.68	$2.16 \cdot 10^{-14}$	0.54	$8.44 \cdot 10^{-15}$	0.53	$5.80 \cdot 10^{-15}$
$m = 10$	1.44	$8.42 \cdot 10^{-14}$	1.09	$2.51 \cdot 10^{-14}$	1.08	$2.32 \cdot 10^{-14}$
$m = 20$	3.65	$9.56 \cdot 10^{-13}$	2.51	$1.27 \cdot 10^{-13}$	2.72	$1.24 \cdot 10^{-13}$
$m = 50$	15.83	$3.09 \cdot 10^{-12}$	9.67	$1.24 \cdot 10^{-12}$	9.83	$9.07 \cdot 10^{-13}$
$m = 100$	56.33	$2.15 \cdot 10^{-11}$	30.89	$6.02 \cdot 10^{-12}$	32.46	$3.60 \cdot 10^{-12}$

Table 1: CPU time and orthogonality

	Single GS, nn=800		Double GS, nn=800		Triple GS, nn=800	
	time [s]	orth	time [s]	orth	time [s]	orth
$m = 5$	0.68	$2.16 \cdot 10^{-14}$	0.84	$4.17 \cdot 10^{-15}$	0.96	$3.91 \cdot 10^{-15}$
$m = 10$	1.44	$8.42 \cdot 10^{-14}$	2.23	$4.17 \cdot 10^{-15}$	2.55	$3.91 \cdot 10^{-15}$
$m = 20$	3.65	$9.56 \cdot 10^{-13}$	6.18	$1.91 \cdot 10^{-15}$	7.85	$1.54 \cdot 10^{-15}$
$m = 50$	15.83	$3.09 \cdot 10^{-12}$	27.24	$2.15 \cdot 10^{-15}$	38.25	$1.76 \cdot 10^{-15}$
$m = 100$	56.33	$2.15 \cdot 10^{-11}$	99.86	$2.52 \cdot 10^{-15}$	148.13	$2.53 \cdot 10^{-15}$

Table 2: CPU time and orthogonality for MV only

b)

Looking at the results in Table 1, the single GS(for version) is both faster and more accurate compared to single GS (MV version), observing almost a halving in time and one order of magnitude smaller error. The modified GS(for version) compared to single GS (for version), both the accuracy and time are similiar.

Double GS (MV) takes longer time than single GS, but reaches machine precision instantly, as can be seen in Table 2. Triple GS (MV) takes longer time than double, but since Double GS (MV) reached machine precision error level at the same rate, one gains nothing from using triple instead of double in this scenario.

In this setting Double GS (MV) is best for accuracy, Single GS (for version) is fastest.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Problem 4

a)

We want to prove that

$$\mu\boldsymbol{\omega} = (\mathbf{K_m^T K_m})^{-1}\mathbf{K_m^T A K_m}\boldsymbol{\omega} \tag{4}$$

is equivalent to

$$\mu\boldsymbol{\omega} = \mathbf{H_m}\boldsymbol{\omega}. \tag{5}$$

Since $\mathbf{K_m}$ in (4) is invertible, it also has full column rank. We can then rewrite $\mathbf{K_m}$ as

$$\mathbf{K_m} = \mathbf{Q_m R_m}, \tag{6}$$

where $\mathbf{Q_m}$ is an orthogonal matrix and $\mathbf{R_m}$ is an upper triangular matrix. Now we substitute (6) in (4) giving

$$\mu\boldsymbol{\omega} = (\mathbf{R_m^T Q_m^T Q_m R_m})^{-1}\mathbf{R_m^T Q_m^T A Q_m R_m}\boldsymbol{\omega}. \tag{7}$$

Since we know that $\mathbf{Q_m^T Q_m} = \mathbf{I}$, $(\mathbf{R_m^T})^{-1}\mathbf{R_m^T} = \mathbf{I}$ and $\mathbf{H_m} = \mathbf{Q_m^T A Q_m}$, we get

$$\mu\boldsymbol{\omega} = (\mathbf{R_m^T R_m})^{-1}\mathbf{R_m^T Q_m^T A Q_m R_m}\boldsymbol{\omega} = \mathbf{R_m^{-1}}(\mathbf{R_m^T})^{-1}\mathbf{R_m^T Q_m^T A Q_m R_m}\boldsymbol{\omega} = \mathbf{R_m^{-1} H_m R_m}\boldsymbol{\omega}. \tag{8}$$

Multiplying both sides in (8) by $\mathbf{R_m}$, we get

$$\mathbf{R_m}\mu\boldsymbol{\omega} = \mathbf{H_m R_m}\boldsymbol{\omega} \tag{9}$$

$$\mathbf{y} = \mathbf{R_m}\boldsymbol{\omega} \Rightarrow \mu\mathbf{y} = \mathbf{H_m y} \tag{10}$$

This is the eigenvalue problem $\mathbf{H_m y} = \mu\mathbf{y}$ solved by the Arnoldi method. Therefore, equation (4) produces identical eigenvalue approximations to Arnoldi's method. □

b)

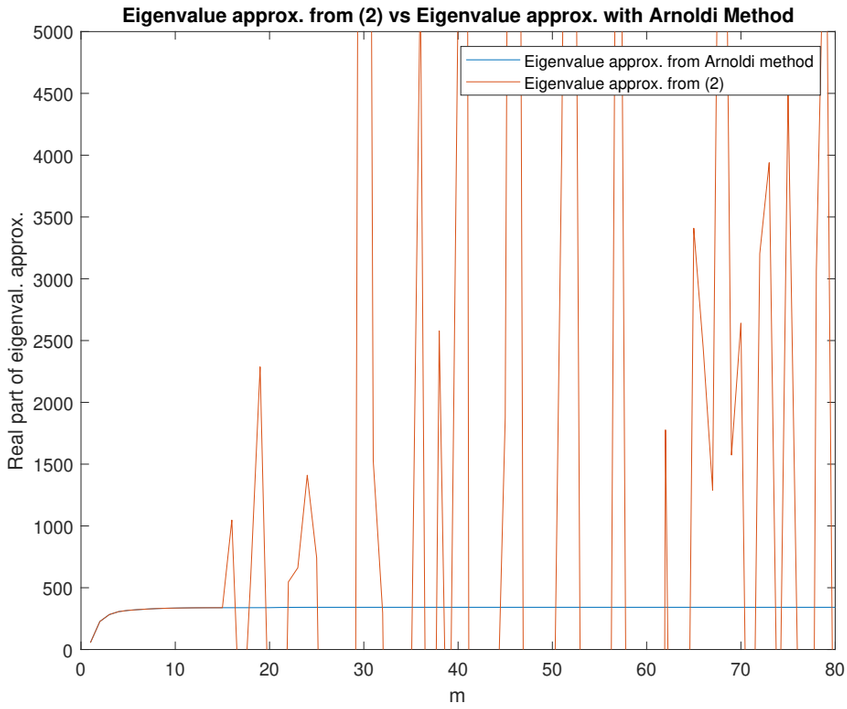


Figure 5: Comparison between eigenvalue approximations from (2) and regular AM.

c)

While both methods are mathematically equivalent as we saw in part a), they differ in numerical behaviour. In Figure 5 we observe that Arnoldi method converges nicely while the approximation from (2) in the problem text (equation (4) in this report), becomes unstable. This is a consequence from amplification of rounding errors since no orthogonalization is applied with this approach. The Arnoldi method is therefore clearly preferred in this case. Although, worth to mention is that we can see that solving the eigenvalue problem of (4) is numerically faster than the Arnoldi Method.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Problem 5

a) + b)

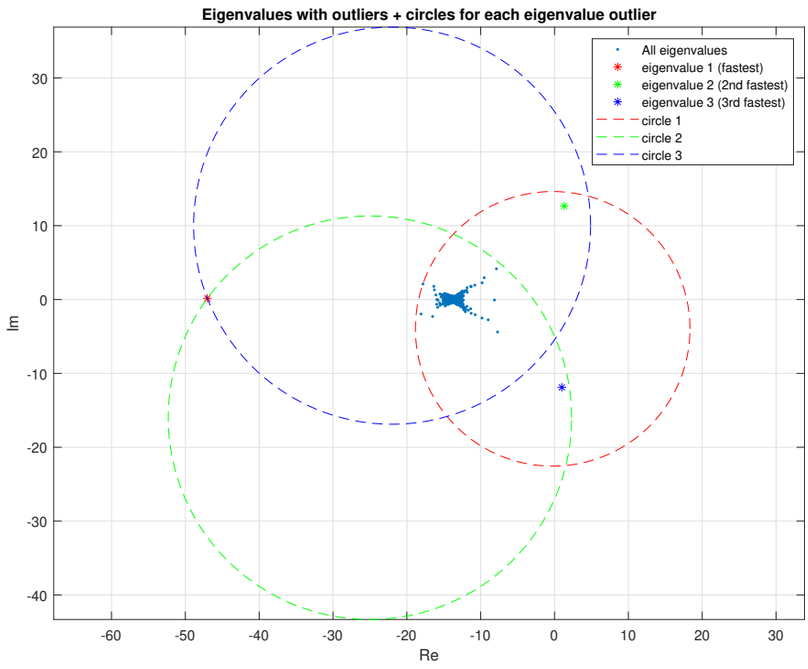


Figure 6: Plot of eigenvalues with marked outliers and circles

Estimated convergence factors: $\alpha_1 = 0.3959$ $\alpha_2 = 0.7020$ $\alpha_3 = 0.8467$. This is done by $\alpha = \frac{\rho}{|\lambda - c|}$ where ρ is the radius of the disk, c is the center of the disk and λ the eigenvalue excluded from the disk according to Corollary 1.4.3 Block 1 and lecture notes.

c)

We expect the fastest convergence to the eigenvalues that are the biggest outliers, marked in Figure 6. The eigenvalue error reaches 10^{-10} when $m = 20$.

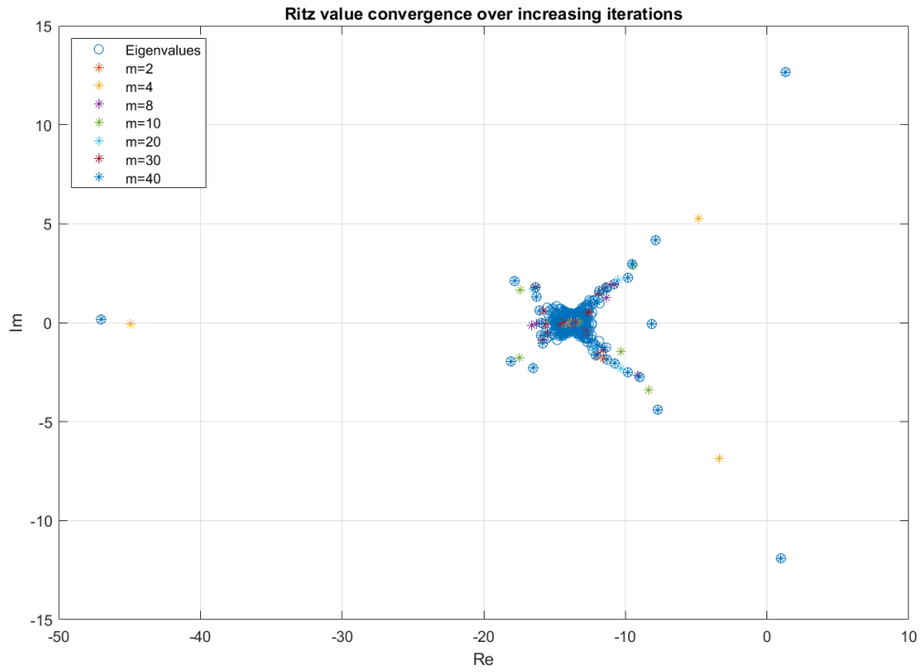


Figure 7: Ritz value convergence over increasing iterations

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

e) + d)

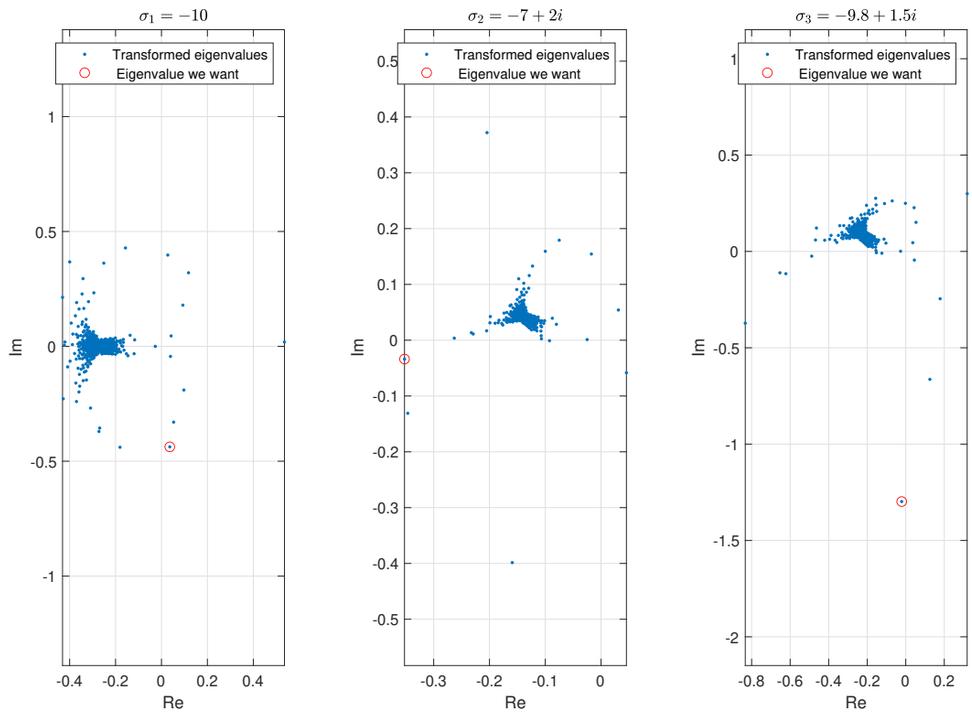


Figure 8: Plot of transformed eigenvalues with different shift factors.

m	Standard AM	$\sigma_1 = -10$	$\sigma_2 = -7 + 2i$	$\sigma_3 = -9.8 + 1.5i$
$m = 10$	2.5	$1.80 \cdot 10^{-3}$	$1.10 \cdot 10^{-3}$	$1.24 \cdot 10^{-5}$
$m = 20$	0.6	$4.88 \cdot 10^{-5}$	$1.15 \cdot 10^{-8}$	$1.15 \cdot 10^{-13}$
$m = 30$	$2 \cdot 10^{-4}$	$5.92 \cdot 10^{-9}$	$1.33 \cdot 10^{-14}$	$9.14 \cdot 10^{-14}$

Table 3: Error for Shift-and-Invert AM for different m and shift values.

When the shift moves closer to the actual eigenvalue we are targeting, $\lambda \approx -9.812 + 2.270i$, the error becomes smaller, as can be seen in Table 3. We can also see in Figure 8 that the closer the shift is to the eigenvalue we want, the further away this eigenvalue is from the others meaning we will converge towards this eigenvalue faster. This shows that a well chosen σ in Shift-and-invert Arnoldi is important.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Problem 6

- **(a) 101 Block 1 applications**

  In this video we learn how large and sparse eigenvalue problems can arise in exploratory data analytics. When dealing with large unordered datasets where the structure is unknown, we want to find patterns. This approach has proven successful in fraud detection (e.g., Panama Papers), network analysis, and climate science.

  The method itself is to tranform the data into a graph and consequently a Laplacian matrix. The columns and rows represent nodes, and a non zero element is an edge between nodes. This matrix is also often large and sparse. By computing the eigenvalues and eigenvectors of the Laplacian matrix, this tells us about the hidden clusters. For example, analyzing the eigenvalues of the MNIST handwritten digit dataset, with this method, reveals distinct groupings.

- **(b) 140 Rayleigh-Ritz method**

  The Rayleigh-Ritz (RR) method can be implemented in scenarios where one would want to approximate eigenvalues for a matrix instead of solving for the eigenvalues of this matrix directly.

  Given an orthogonal matrix $\mathbf{Q} = \begin{bmatrix}\mathbf{q}_1 \cdots \mathbf{q}_k\end{bmatrix} \in \mathbb{R}^{n\times k}$, the idea of the RR-method is to view span$\{\mathbf{q}_1 \cdots \mathbf{q}_n\}$ as a search subspace to look for approximate solutions. We can look for these approximate solutions by letting the eigenvector $\mathbf{x} \approx \mathbf{Qz}$ for some vector $\mathbf{z} \in \mathbb{R}^k$, giving $\mathbf{Ax} = \lambda\mathbf{x} \quad \Leftrightarrow \quad \mathbf{AQz} \approx \lambda\mathbf{Qz}$. Using that $\mathbf{Q}$ is an orthogonal matrix and using impose relation, we get $\mathbf{Bz} = \tilde{\lambda}\mathbf{z}$ where $\mathbf{B} = \mathbf{Q^T AQ}$ and solving this eigenvalue problem for B gives the approximate eigenvalues $\tilde{\lambda}$. If one wants the eigenvector approximations, since $\mathbf{z}$ contains the linear combination coefficients for $\mathbf{Q}$, one has to multiply this $\mathbf{z}$ by $\mathbf{Q}$. If the eigenvectors approximately lie in the search subspace, the RR-method will often give good approximations for the eigenvalues.

  Worth to note is also that if $k << n$, it is cheaper to compute the eigenvalues of $\mathbf{B} \in \mathbb{R}^{k\times k}$ rather than computing the eigenvalues for the original matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$

- **(c) 169 Breakdown**
  Breakdown for the Arnoldi method occurs when trying to divide by zero. More specifically when $\beta = 0$ for the Gram-Schmidt orthogonalization, this also implies that the last element of the matrix H_m is $h_{m+1,m} = 0$.

  This in fact means that the Ritz values are exact. This is a direct consequence of the following Lemma:

  If Q_{m+1} and H_m satisfy Arnoldi factorization and $\mu z = H_m z$, then

  $$|| \, Av - \mu v \, || = | \, h_{m+1,m} \, | \cdot | \, e_m^T z \, | \tag{11}$$

  Where $v = Q_m z$ is the eigenvector approximation corresponding to μ. And since $h_{m+1,m}$ in RHS is zero, the residual of the LHS is zero, which means we have a zero approximation error.

- **(d) 168 Shift-and-invert Arnoldi method for eigenvalue problems**

  Similar to the relationship between the Power Method and the Inverse Iteration, the Shift-and-Invert Arnoldi Method is a method to converge to eigenvalues for a matrix A which are close to a given shift μ by using the AM for $\mathbf{B} = (\mathbf{A} - \mu\mathbf{I})^{-1}$. The downside with the Shift-and-Invert AM is that we are not always certain which eigenvalue we converge to, but the upside is that we often have faster convergence to the eigenvalues close to the shift compared to the II method.

  For the Shift-and-Invert AM, the idea is to apply the AM on $(\mathbf{A} - \mu\mathbf{I})^{-1}\mathbf{b}$ using only matrix vector products, meaning one does not have to explicitly compute the inverse. In the code for the AM, to change it to the Shift-and-Invert AM, one has to first introduce μ as an argument and thereafter define the identity matrix as a large and sparse identity matrix since the Shift-and-Invert AM is applied to large and sparse problems. The matrix vector product in the for loop then also has to be changed from $\mathbf{w} = \mathbf{AQ}(:, k)$ to $\mathbf{w} = (\mathbf{A} - \mu\mathbf{I})\backslash\mathbf{Q}(:, k)$.

  Since we are interested in the eigenvalues of A, we also have to reverse the transformation when we extract the ritz values, this is done by changing $eig(\mathbf{H}(1:m, 1:m))$ to $1./(eig(\mathbf{H}(1:m, 1:m))\text{-}\mu)$.

  To solve the linear system faster, one can also precompute for example LU-factorization for $(A - \mu I)$ giving w = U\(L\Q(:,k)).

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Problem 8

b)

$\mathbf{C} \in \mathbb{R}^{4096\times 4096}$ and $\mathbf{X_c} \in \mathbb{R}^{4096\times 400}$

(1) Forming $\mathbf{C} = \mathbf{X_c}\mathbf{X_c^T}$ takes 0.079744 seconds.
(2) Computing $\mathbf{Cb}$ for 100 different random start vectors $\mathbf{b}$ takes 0.435369 seconds.
(3) Computing $\mathbf{X_c}(\mathbf{X_c^T}\mathbf{b})$ for 100 different random $\mathbf{b}$ takes 0.062413 seconds.

c)

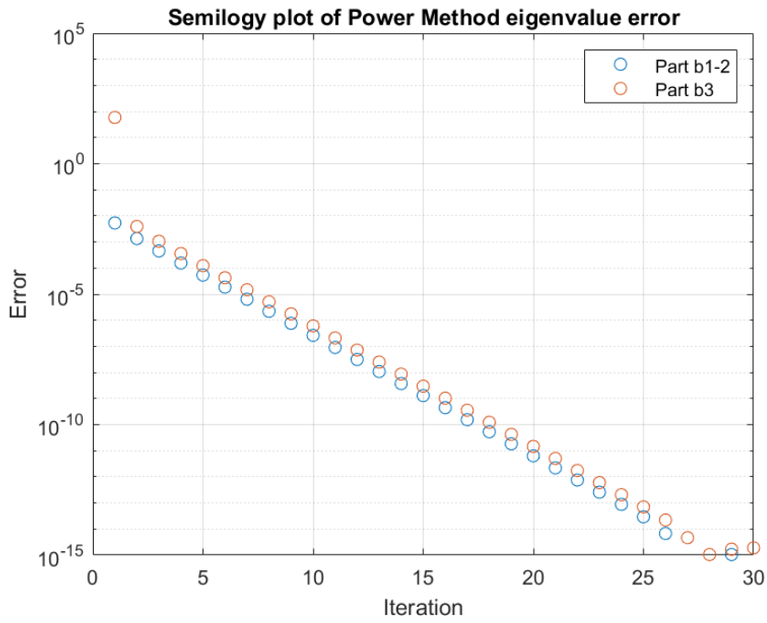


Figure 9: Comparison of eigenvalue error for two approaches.

The time for method b.1-2 is 0.269 seconds and the time for method b.3 is 0.021 seconds. We can therefore draw the conclusion that, since the convergences are almost identical, it is better to use method b.3 here.

Method	Time [s]
b.1-2	0.269
b.3	0.021

Table 4: Time comparison

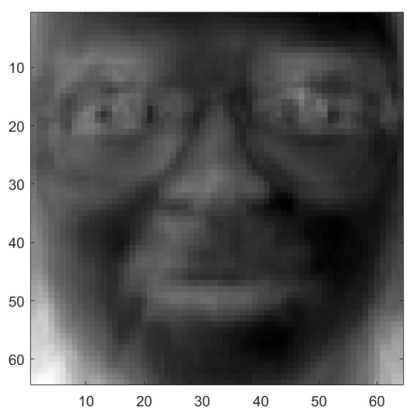


Figure 10: Eigenface corresponding to largest eigenvalue.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

d)

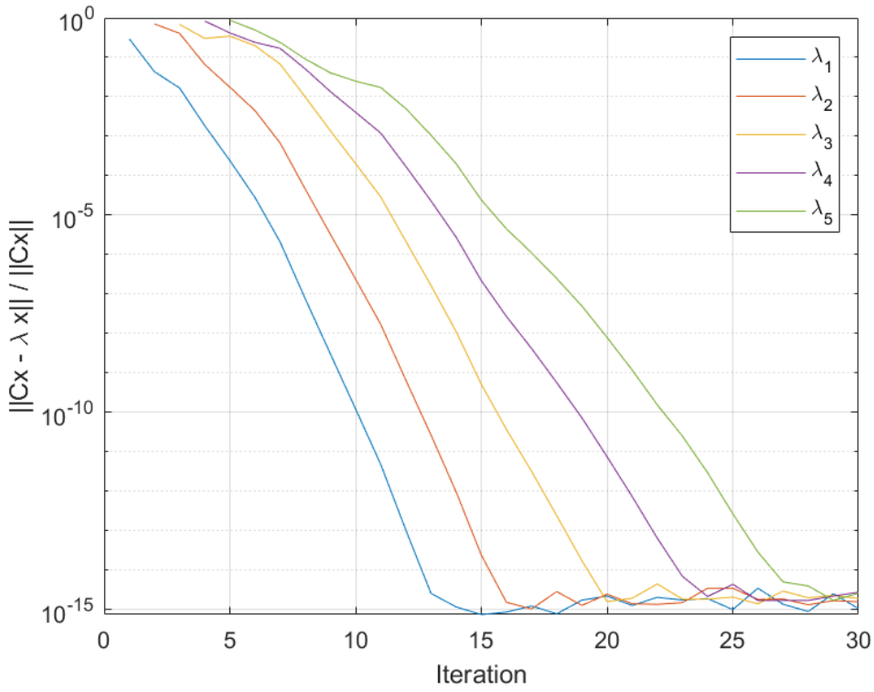


Figure 11: Error of 5 biggest eigenvalues of H.

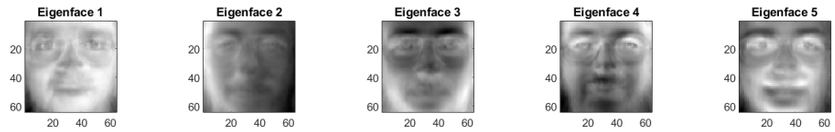


Figure 12: Eigenfaces corresponding to 5 largest eigenvalues.

Some of the eigenfaces seem to be inverted.

e)

The solution from Arnoldi method in d) takes ≈ 0.51 s which is longer than any of the two methods of PM used in c). However this comparison is misleading since Arnoldi computes all 5 largest eigenvalues at the same time. In the scenario when one want to find only the eigenface with the largest eigenvalue, the Power Method accurately finds this eigenvalue if the method converges. Using the Arnoldi Method, we can approximate several large eigenvalues at the same time but due to the fact that an eigenvector $\mathbf{v}$ is equivalent to $-\mathbf{v}$ for eigenvector problems, the image might end up inverted in contrast which can be seen comparing the first eigenface in Figure 12 and the eigenface in Figure 10.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

f)

Face 5 is the scariest.

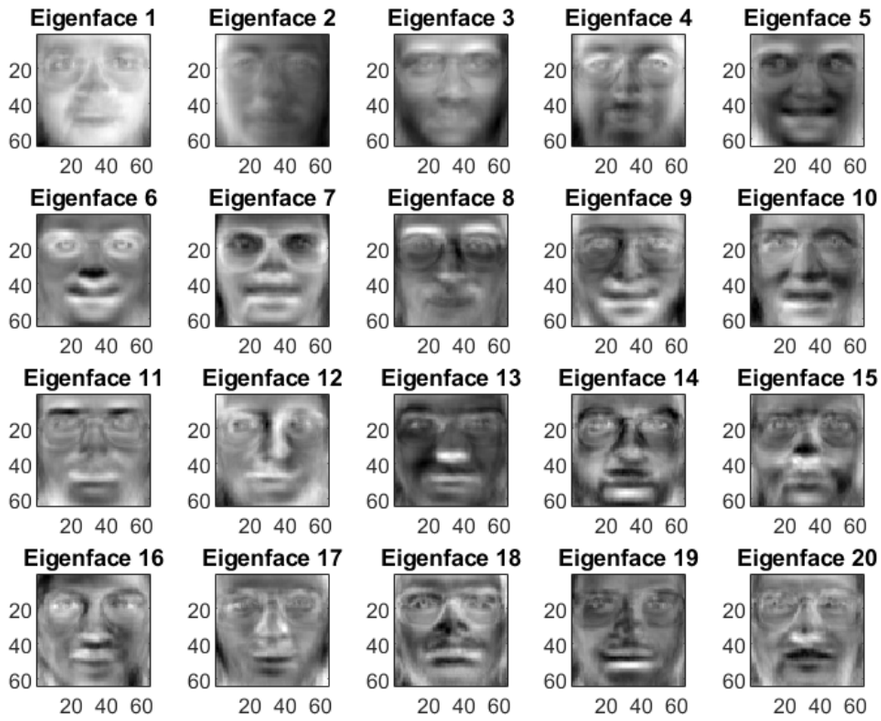


Figure 13: Eigenfaces corresponding to 20 first eigenvalues.